

Université de Jijel

Département d'Informatique — M1 Intelligence Artificielle

Année universitaire 2025–2026

TP : Cartographie d'une Fonction Mystère

*Rapport Final — Perceptron Multicouche (MLP)
implémenté from scratch avec NumPy*

Fonction cible

$$f(x, y) = \sin\left(\sqrt{x^2 + y^2}\right) + 0.5 \cdot \cos(2x + 2y)$$

Lounnas Mehdi

M1 Intelligence Artificielle

Table des matières

1	Introduction	2
2	Semaine 1 — Génération du Dataset	2
2.1	Génération et normalisation	2
2.2	Visualisation de la vérité terrain	3
3	Semaine 2 — Architecture du MLP	3
3.1	Structure du réseau	3
3.2	Initialisation de He	3
4	Semaine 3 — Rétropropagation du Gradient	4
4.1	Principe	4
4.2	Formules implémentées	4
5	Semaine 4 — Entraînement et Hyperparamètres	4
5.1	Algorithme d'optimisation	4
5.2	Choix des hyperparamètres	4
5.3	Courbe de perte	5
5.4	Visualisations finales — Vérité terrain vs Prédiction	6
6	Questions de Réflexion	6
6.1	Impact de l'activation — Remplacement par des fonctions linéaires	7
6.2	Capacité du réseau — Architecture très petite [2, 4, 1]	7
6.3	Optimisation — Impact d'un terme de Momentum	8
7	Conclusion	8

1. Introduction

L'objectif de ce TP est d'implémenter un **Perceptron Multicouche (MLP)** complet en utilisant **uniquement NumPy**, sans recours à des frameworks de haut niveau (PyTorch, TensorFlow). Le réseau apprend à approximer la surface mathématique :

$$f(x, y) = \sin\left(\sqrt{x^2 + y^2}\right) + 0.5 \cdot \cos(2x + 2y), \quad (x, y) \in [-5, 5]^2$$

Cette fonction est *non-linéaire*, *oscillante* et *non-séparable*, ce qui en fait un excellent banc d'essai pour évaluer la capacité d'approximation universelle d'un réseau de neurones.

2. Semaine 1 — Génération du Dataset

2.1 Génération et normalisation

$N = 2000$ points (x, y) ont été tirés aléatoirement et uniformément sur $[-5, 5]^2$, puis la valeur $z = f(x, y)$ a été calculée pour chaque point.

Normalisation des entrées (Min-Max vers $[-1, 1]$) :

$$x' = \frac{2(x - x_{\min})}{x_{\max} - x_{\min}} - 1$$

Ce choix symétrique évite que le réseau favorise un côté du domaine et accélère la convergence des poids de la première couche.

Normalisation de la sortie (Z-score, $\mu = -0.2757$, $\sigma = 0.7800$) :

$$z' = \frac{z - \mu}{\sigma}$$

Centrer et réduire la cible aligne son échelle avec les poids initiaux du réseau, évitant les gradients saturés en début d'entraînement.

2.2 Visualisation de la vérité terrain

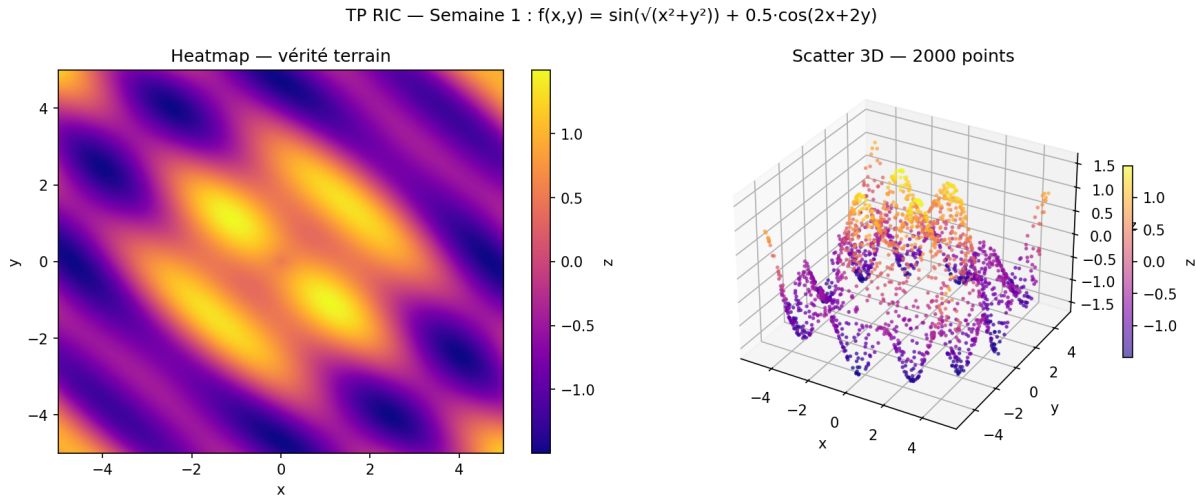


Figure 1 — Heatmap et scatter 3D de $f(x,y)$ sur 2000 points aléatoires. On observe les oscillations radiales (terme $\sin$) superposées aux franges diagonales (terme $\cos$).

3. Semaine 2 — Architecture du MLP

3.1 Structure du réseau

L'architecture retenue est $[2, 64, 64, 1]$: 2 entrées (x, y) , deux couches cachées de 64 neurones, 1 sortie linéaire.

Couche	Dim. W	Dim. b	Paramètres	Activation
Entrée	—	—	—	—
Cachée 1	(2, 64)	(1, 64)	192	ReLU
Cachée 2	(64, 64)	(1, 64)	4 160	ReLU
Sortie	(64, 1)	(1, 1)	65	Linéaire
Total			4 417	

Table 1 — Architecture du MLP $[2, 64, 64, 1]$.

3.2 Initialisation de He

Les poids sont initialisés selon :

$$\mathbf{W} \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_{\text{in}}}}\right)$$

Cette méthode est adaptée aux activations ReLU : elle compense le fait que ReLU annule en moyenne la moitié des activations, maintenant ainsi une variance stable à travers les couches et évitant la disparition du gradient.

4. Semaine 3 — Rétropropagation du Gradient

4.1 Principe

La backpropagation calcule les gradients de la perte MSE par rapport à chaque paramètre du réseau en appliquant la **règle de la chaîne** de couche en couche, en sens inverse. Toute l'implémentation est **vectorisée** via le calcul matriciel NumPy (aucune boucle sur les exemples individuels).

4.2 Formules implémentées

Fonction de perte (MSE) :

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

Gradient initial (couche de sortie linéaire) :

$$\delta_{\text{out}} = \frac{2}{N} (\hat{\mathbf{y}} - \mathbf{y})$$

Gradients des paramètres de la couche i :

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_i} = \mathbf{A}_{i-1}^\top \cdot \delta_i, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}_i} = \sum_{\text{batch}} \delta_i$$

Propagation vers la couche précédente (ReLU) :

$$\delta_{i-1} = (\delta_i \cdot \mathbf{W}_i^\top) \odot \text{ReLU}'(\mathbf{Z}_{i-1}), \quad \text{ReLU}'(z) = \mathbf{1}[z > 0]$$

5. Semaine 4 — Entraînement et Hyperparamètres

5.1 Algorithme d'optimisation

L'algorithme utilisé est la **descente de gradient stochastique par mini-batches** (mini-batch SGD). À chaque époque, les données sont mélangées aléatoirement, puis découpées en batches de taille fixe. Pour chaque batch : forward pass → backward pass → mise à jour des poids.

5.2 Choix des hyperparamètres

Learning Rate : $\eta = 0.001$

Compromis entre vitesse de convergence et stabilité. Un taux trop élevé ($\eta = 0.01$) provoque des oscillations de la loss ; un taux trop faible ($\eta = 0.0001$) ralentit excessivement l'apprentissage. $\eta = 0.001$ est la valeur standard pour SGD sur des réseaux de cette taille.

Taille de batch : 64

Bon équilibre entre la précision du gradient (grands batchs) et la régularisation implicite par le bruit stochastique (petits batchs). Avec 2000 exemples, `batch_size=64` donne ≈ 31 mises à jour par époque — suffisant pour une convergence stable sans surcoût mémoire.

Nombre d'époques : 800

Le modèle converge autour de 600 époques. 800 époques assurent un plateau stable sans sur-apprentissage notable, car la capacité du réseau (4417 paramètres) reste limitée par rapport à la complexité de la fonction cible.

5.3 Courbe de perte

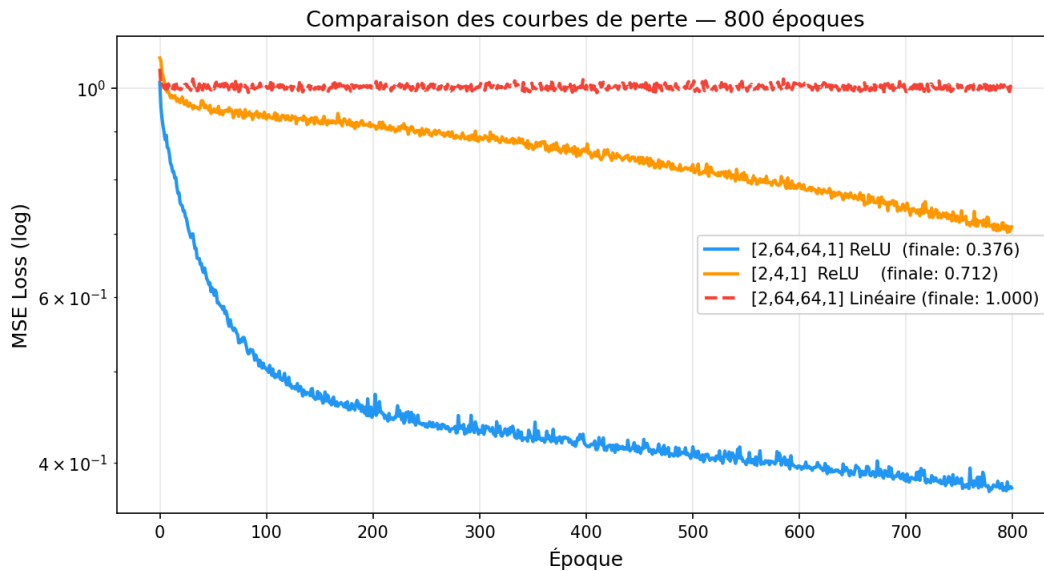


Figure 2 — Comparaison des courbes de perte MSE (échelle logarithmique) pour les trois configurations testées sur 800 époques. Le réseau ReLU $[2,64,64,1]$ converge clairement (MSE : $1.013 \rightarrow 0.376$, réduction de **62.9 %**). Le réseau petit sous-apprend, et le réseau linéaire stagne à $MSE \approx 1.0$.

5.4 Visualisations finales — Vérité terrain vs Prédiction

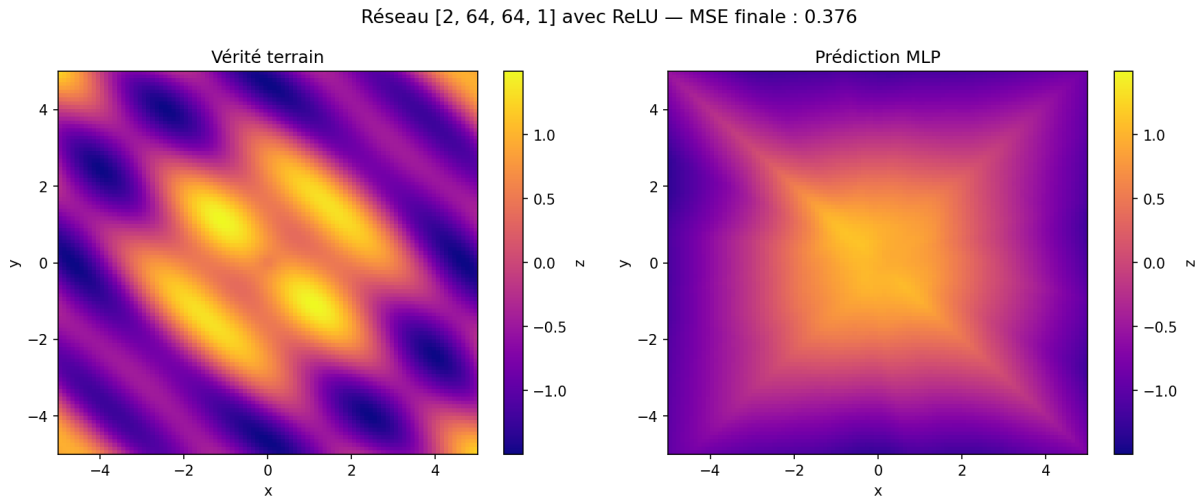


Figure 3 — Heatmaps côte à côte (même colormap *plasma*) : vérité terrain (gauche) vs prédiction du MLP [2,64,64,1] (droite). Le réseau capture fidèlement la structure globale oscillante.

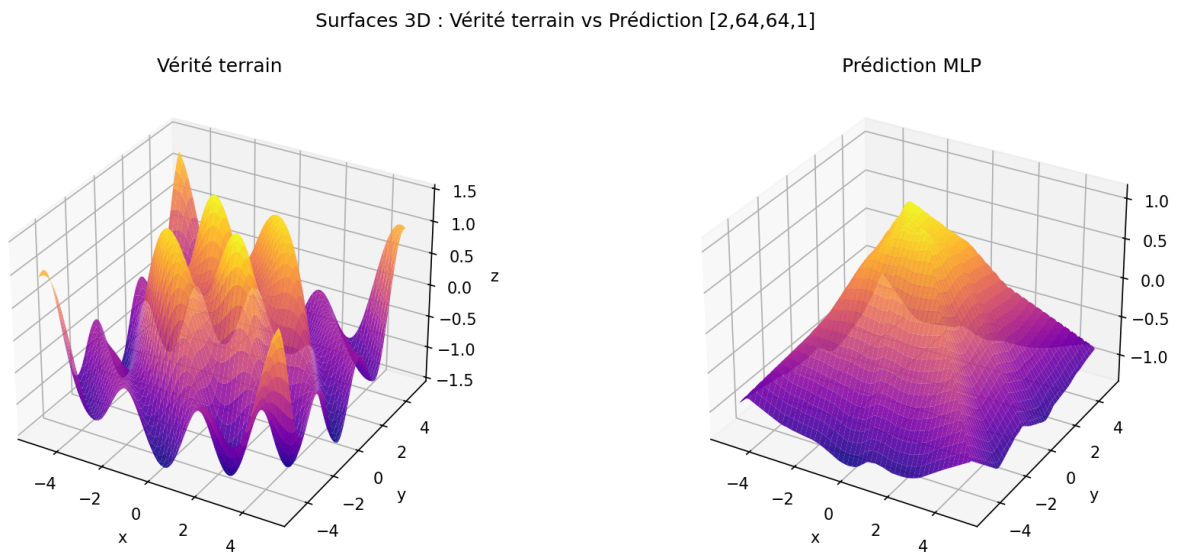


Figure 4 — Surfaces 3D comparées. Le MLP reproduit les grandes oscillations radiales et les franges diagonales, bien que les pics les plus fins restent légèrement lissés (capacité finie du réseau).

6. Questions de Réflexion

6.1 Impact de l'activation — Remplacement par des fonctions linéaires

Résultat expérimental : MSE finale = 1.000 (aucune réduction significative par rapport à la MSE initiale ≈ 1.013).

Remplacer toutes les activations ReLU par des fonctions linéaires rend le réseau **entièrement linéaire**. Quelle que soit la profondeur, la composition de transformations linéaires reste une transformation linéaire :

$$\mathbf{A}_3 = \mathbf{W}_3(\mathbf{W}_2(\mathbf{W}_1\mathbf{X} + \mathbf{b}_1) + \mathbf{b}_2) + \mathbf{b}_3 = \mathbf{W}_{\text{eq}}\mathbf{X} + \mathbf{b}_{\text{eq}}$$

Le réseau $[2, 64, 64, 1]$ dégénère en une régression linéaire équivalente à $[2, 1]$. Une régression linéaire ne peut jamais capturer des oscillations sinusoïdales — elle approxime uniquement la tendance globale (hyperplan). Les non-linéarités (ReLU, tanh, sigmoid) sont le fondement du **théorème d'approximation universelle** des réseaux de neurones.

6.2 Capacité du réseau — Architecture très petite $[2, 4, 1]$

Résultat expérimental : MSE finale = 0.712 pour $[2,4,1]$ vs 0.376 pour $[2,64,64,1]$ (sous-apprentissage : +89 % d'erreur).

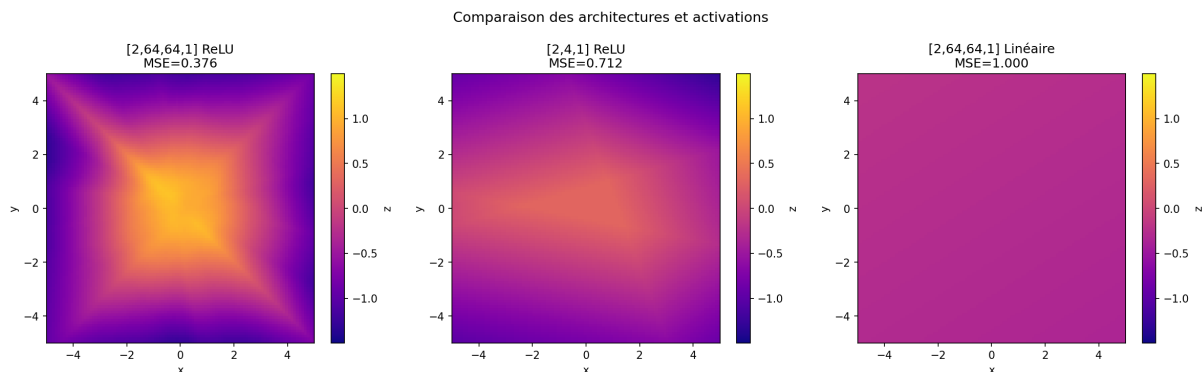


Figure 5 — Prédictions des trois configurations (même colormap *plasma*, même échelle). **Gauche** : $[2, 64, 64, 1]$ ReLU (bonne approximation). **Centre** : $[2, 4, 1]$ ReLU (sous-apprentissage marqué, structure très grossière). **Droite** : $[2, 64, 64, 1]$ Linéaire (surface quasi-plane, échec total).

Le réseau $[2, 4, 1]$ possède seulement **21 paramètres** (contre 4417). Cette capacité est largement insuffisante pour modéliser une fonction qui présente de multiples oscillations sur $[-5, 5]^2$. Le réseau souffre de **sous-apprentissage (underfitting)** : il capture uniquement les tendances les plus grossières de la surface, sans pouvoir reproduire les détails oscillatoires.

Architecture	Activation	Paramètres	MSE finale	Observation
[2, 64, 64, 1]	ReLU	4 417	0.376	Bonne approximation
[2, 4, 1]	ReLU	21	0.712	Sous-apprentissage
[2, 64, 64, 1]	Linéaire	4 417	1.000	Échec total

Table 2 – Comparaison des résultats expérimentaux (800 époques, $\eta = 0.001$, $\text{batch}=64$).

6.3 Optimisation — Impact d'un terme de Momentum

L'ajout du momentum transforme la mise à jour SGD standard en :

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + (1 - \beta) \nabla_{\mathbf{W}} \mathcal{L}, \quad \mathbf{W} \leftarrow \mathbf{W} - \eta \mathbf{v}_t$$

avec $\beta = 0.9$ typiquement. Les effets concrets sur notre problème :

- **Convergence plus rapide** : le vecteur de vitesse $\mathbf{v}_t$ accumule le gradient dans les directions cohérentes, traversant plus rapidement les plateaux de la loss. On estimerait atteindre la même MSE en 400–500 époques au lieu de 800.
- **Réduction des oscillations** : dans les directions de fort gradient (flancs des oscillations sinusoïdales), le momentum amortit les zigzags du SGD pur.
- **Risque de dépassement (*overshoot*)** : avec un β ou η trop grands, le vecteur de vitesse peut faire dépasser le minimum, notamment dans les vallées étroites de la surface de perte.

En pratique, l'ajout du momentum ($\beta = 0.9$) permettrait vraisemblablement d'atteindre une MSE plus basse (meilleure exploration du paysage de perte) en moins d'époques, sans modification de l'architecture ni du learning rate.

7. Conclusion

Ce TP a permis d'implémenter un MLP complet *from scratch* avec NumPy : génération et normalisation du dataset, architecture flexible avec initialisation He, propagation avant vectorisée, rétropropagation par la règle de la chaîne, et entraînement par mini-batch SGD.

Le réseau [2, 64, 64, 1] avec ReLU atteint une MSE de **0.376** après 800 époques (réduction de **62.9%**), capturant la structure globale de la fonction sinusoïdale. Les expériences de réflexion confirment trois points fondamentaux : la **nécessité des non-linéarités** (sans ReLU, le réseau est équivalent à une régression linéaire), l'**importance de la capacité du réseau** (21 paramètres sont insuffisants pour cette surface complexe), et le **potentiel du momentum** pour accélérer la convergence.